MKR

December 15, 2025

[208]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.linear_model import Perceptron as Perceptron_S
from sklearn.model_selection import cross_val_score
```

[209]:
```
data = {
    'A': [0, 0, 1, 1],
    'B': [0, 1, 0, 1],
    'y': [1, 1, 1, 0]
}
```

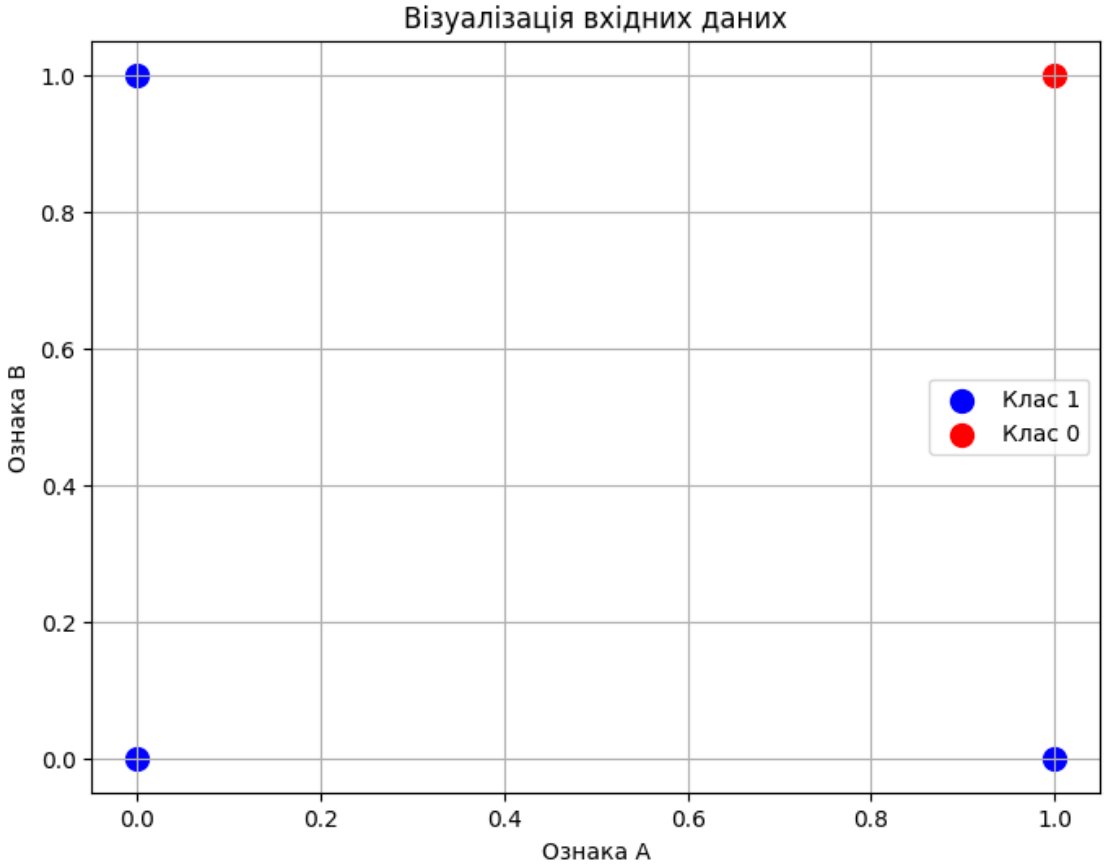
[210]:
```
data = pd.DataFrame(data)
data
```

[210]:

	A	B	y
0	0	0	1
1	0	1	1
2	1	0	1
3	1	1	0

[211]:
```
plt.figure(figsize=(8, 6))

plt.scatter(data[data['y'] == 1]['A'],
            data[data['y'] == 1]['B'],
            color='blue',
            label='   1',
            marker='o',
            s=100)
plt.scatter(data[data['y'] == 0]['A'],
            data[data['y'] == 0]['B'],
            color='red',
            label='   0',
            marker='o', s=100)

plt.title('               ')
```

```
plt.xlabel('    A')
plt.ylabel('    B')
plt.legend()
plt.grid(True)
plt.show()
```



[212]:
```
X = data[['A', 'B']].to_numpy()
```

[213]:
```
y = data['y'].to_numpy()
y[y == 0] = -1
```

[214]:
```
class Perceptron():
  def __init__(self, learning_rate=0.01, n_iters=50):
      self.learning_rate = learning_rate
      self.n_iters = n_iters
      self.weights = None

  def fit(self, X, y):
    n_samples, n_features = X.shape
```

```
    X_b = np.hstack((X, np.ones((n_samples, 1))))
    self.weights = np.zeros(n_features+1)
    X_prime = X_b * y[:, np.newaxis]
    for _ in range(self.n_iters):
      np.random.shuffle(X_prime)
      for i in X_prime:
        if np.dot(self.weights, i) <= 0:
          self.weights += self.learning_rate * i

  def predict(self, X):
    n_samples = X.shape[0]
    X_b = np.hstack((X, np.ones((n_samples, 1))))
    y_predicted = np.where(np.dot(X_b, self.weights) >= 0, 1, -1)
    return y_predicted

  def score(self, X, y):
    y_pred = self.predict(X)
    return accuracy_score(y, y_pred)

  def get_params(self, deep=True):
    return {"learning_rate": self.learning_rate, "n_iters": self.n_iters}

  def set_params(self, **params):
    for param, value in params.items():
      setattr(self, param, value)
    return self
```

[215]:
```
percepton_test = Perceptron()

percepton_test.fit(X, y)
```

[216]:
```
W = percepton_test.weights
W
```

[216]:
```
array([-0.02, -0.02,  0.03])
```

[217]:
```
def print_computation(A, B, y_true, w0, w1, w2):

    z =  (w0 * A) + (w1 * B) + w2

    prediction = 1 if z >= 0 else 0

    result_status = "     " if prediction == y_true else "    "

    print(f"---              (A={A}, B={B}) ---")
    print(f"       : z = w0*A + w1*B + w2")
    print(f"         : z =  ({w0} * {A}) + ({w0} * {B}) + {w2} = {z}")
```

```
    print(f"          :      z = {z} ({'>=' if z >= 0 else '<'} 0),        =␣
↪{prediction}")
    print(f"     :      ({prediction}) ==            ({y_true}) ->␣
↪{result_status}")
    print("-" * 45 + "\n")
```

[218]:
```
w0, w1, w2 = W[0], W[1], W[2]
```

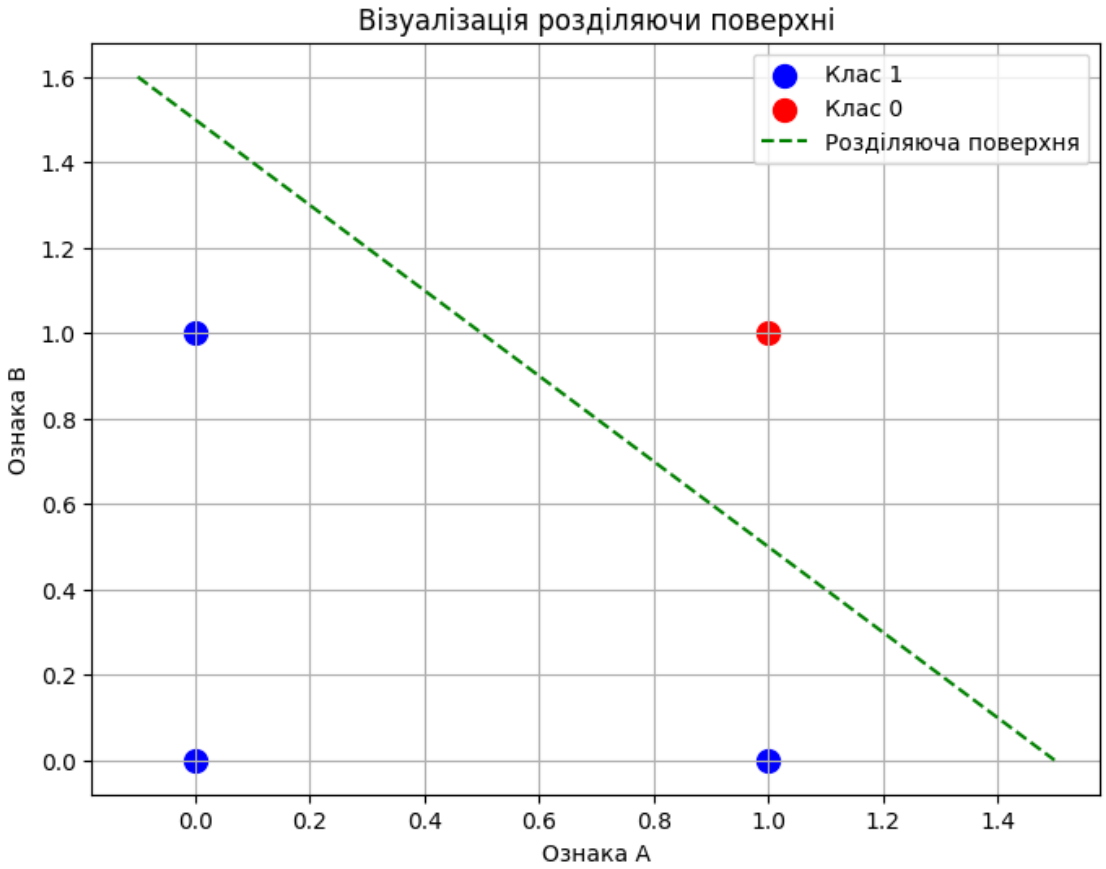
[222]:
```
print_computation(0, 0, 1, w0, w1, w2)
print_computation(0, 1, 1, w0, w1, w2)
print_computation(1, 0, 1, w0, w1, w2)
print_computation(1, 1, 0, w0, w1, w2)
```

```
---              (A=0, B=0) ---
       : z = w0*A + w1*B + w2
         : z =  (-0.02 * 0) + (-0.02 * 0) + 0.03 = 0.03
          :      z = 0.03 (>= 0),        = 1
     :      (1) ==             (1) ->
---------------------------------------------

---              (A=0, B=1) ---
       : z = w0*A + w1*B + w2
         : z =  (-0.02 * 0) + (-0.02 * 1) + 0.03 = 0.010000000000000002
          :      z = 0.010000000000000002 (>= 0),        = 1
     :      (1) ==             (1) ->
---------------------------------------------

---              (A=1, B=0) ---
       : z = w0*A + w1*B + w2
         : z =  (-0.02 * 1) + (-0.02 * 0) + 0.03 = 0.009999999999999998
          :      z = 0.009999999999999998 (>= 0),        = 1
     :      (1) ==             (1) ->
---------------------------------------------

---              (A=1, B=1) ---
       : z = w0*A + w1*B + w2
         : z =  (-0.02 * 1) + (-0.02 * 1) + 0.03 = -0.009999999999999995
          :      z = -0.009999999999999995 (< 0),        = 0
     :      (0) ==             (0) ->
---------------------------------------------
```

$w_0 \cdot x_0 + w_1 \cdot x_1 + w_2 = 0$
$ x_1 = (- w_2 - w_0 x_0) / w_1 $

[220]:
```
x_line = np.linspace(-0.1, 1.5, 100)
y_line = -(w0 * x_line + w2) / w1
```

[221]:
```
plt.figure(figsize=(8, 6))

plt.scatter(data[data['y'] == 1]['A'],
            data[data['y'] == 1]['B'],
            color='blue',
            label='   1',
            marker='o',
            s=100)
plt.scatter(data[data['y'] == -1]['A'],
            data[data['y'] == -1]['B'],
            color='red',
            label='   0',
            marker='o', s=100)
plt.plot(x_line, y_line, color='green', linestyle='--', label='           ')
plt.title('                  ')
plt.xlabel('    A')
plt.ylabel('    B')
plt.legend()
plt.grid(True)
plt.show()
```



[221]: